Licence de Sciences et Technologies — Université de Perpignan Via Domitia
S3 – mention Informatique — V. Zucca

Programmation en `C`

TP3 : Types de données composés

Exercice 1. Dans cet exercice, nous allons réaliser quelques traitements simples sur des tableaux d'entiers.

1. Écrire un programme qui demande à l'utilisateur de saisir les éléments d'un tableau d'entiers dont il choisira la taille.
2. Écrire une fonction `int array_min(int *, unsigned int)` qui renvoie la plus petite valeur contenue dans un tableau.
3. Écrire une fonction `int array_max(int *, unsigned int)` qui renvoie la plus grande valeur contenue dans un tableau.
4. Écrire une fonction `int array_find(int *, unsigned int, int)` qui renvoie l'indice de la première occurrence d'une valeur donnée dans le tableau, ou `-1` si la valeur n'est pas présente.
5. Écrire une fonction `float array_average(int *, unsigned int)` qui renvoie la moyenne des valeurs contenues dans le tableau.

Exercice 2. Dans cet exercice, nous allons trier un tableau d'entiers en place, c'est à dire sans utiliser de tableau supplémentaire.

1. Écrire une fonction `void swap(int *, int *)` qui échange les valeurs de deux variables.
2. Écrire une fonction `void array_sort(int *, unsigned int)` qui trie les éléments d'un tableau par ordre croissant. La fonction devra modifier directement le tableau. La fonction procèdera de la façon suivante :
   - on recherche la plus petite valeur parmi les éléments du tableau qui n'ont pas encore été triés
   - on échange la valeur trouvée avec le premier élément de la partie du tableau considérée.

   Par exemple, pour le tableau {`7, 2, 9, 4, 1, 5`}, on commence par rechercher la plus petite valeur dans l'ensemble du tableau. On trouve `1`, que l'on échange avec `7` et on obtient {`1, 2, 9, 4, 7, 5`}

   On recommence ensuite en considérant uniquement les éléments à partir de l'indice 1.
3. Modifier la fonction afin qu'elle trie uniquement les éléments situés entre deux indices donnés, sans modifier les autres éléments.

Exercice 3. Un carré latin d'ordre n est un tableau carré de taille `N`×`N` rempli avec `N` symboles différents, de telle sorte que chaque symbole apparaisse exactement une fois sur chaque ligne et exactement une fois sur chaque colonne. On supposera que les symboles sont des entiers entre 1 et `N`.

Exemple : un carré latin d'ordre 4 :

1	2	3	4
2	3	4	1
3	4	1	2
4	1	2	3

1. Écrire une fonction `display` qui affiche un carré latin d'ordre `N` passé en paramètre sous la forme d'un tableau à deux dimensions

2. Écrire une fonction `is_latin_square` qui indique si un tableau 2D de dimension `N` × `N` passé en paramètre est un carré latin ou non.

   NB : on pourra tester la fonction avec les tableaux suivants

1	2	3
2	3	1
3	1	2

1	2	3
2	3	1
3	2	1

Exercice 4. On souhaite enregistrer les mesures effectuées par plusieurs capteurs au cours du temps. Chaque capteur mesure plusieurs grandeurs physiques : température, pression et humidité, toutes les heures. Les grandeurs mesurées sont définies par l'énumération suivante :

```
enum mesure { TEMPERATURE, PRESSION, HUMIDITE };
```

On représente un relevé de mesure sur une journée par le tableau suivant :

```
#define NB_CAPTEURS 5
#define NB_HEURES 24
#define NB_MESURES 3

double releve[NB_CAPTEURS][NB_HEURES][NB_MESURES];
```

Ainsi, `releve[1][7][TEMPERATURE]` représente la température mesurée par le deuxième capteur à la huitième heure de la journée.

1. Écrire une fonction `init_measurements` qui initialise toutes les mesures d'un relevé à 0.

2. Écrire une fonction `average_sensor_measure` qui, sur un relevé donné, renvoie la valeur moyenne d'une grandeur donnée mesurée par un capteur donné au cours de la journée.

3. Écrire une fonction `average_measure` qui, sur un relevé donné, renvoie la valeur moyenne d'une grandeur donnée sur l'ensemble des capteurs au cours de la journée.

4. Écrire une fonction `min_measure` qui renvoie la plus petite valeur mesurée d'une grandeur donnée, par un capteur donné au cours de la journée.

5. Écrire une fonction `max_measure` qui, sur un relevé donné, renvoie la plus grande valeur mesurée d'une grandeur donnée, par un capteur donné au cours de la journée.

6. Écrire une fonction `count_above` qui, sur un relevé donné, renvoie le nombre de mesures d'une grandeur donnée dépassant une valeur seuil donnée.

Exercice 5. Dans cet exercice nous allons coder des traitements élémentaires sur les chaînes de caractères sans utiliser les fonctions du fichier d'entête `string.h`.

1. Coder une fonction `unsigned int string_length(char *)` qui renvoie la longueur d'une chaîne de caractères passée en paramètre. On ne comptera pas le caractère `'\0'` de fin de chaîne, la chaîne vide aura donc une longueur de 0.

2. Coder une fonction `unsigned int char_count(char *, char)` qui renvoie le nombre d'occurences d'un caractère, fourni en paramètre, dans une chaîne de caractères également fournie en paramètre.

3. Coder une fonction `int string_compare(char *, char *)` qui compare deux chaînes de caractères passées en paramètre. La fonction renverra 0 si les deux chaînes sont identiques, -1 si la première chaîne est avant la deuxième dans l'ordre alphabétique et 1 sinon.

4. Écrire une fonction `void string_copy(char *, char *)` qui copie une chaîne de caractères passée en paramètre dans une seconde chaîne de caractères, elle aussi passée en paramètre.

5. Écrire une fonction `void delete_char(char *, char)` qui supprime la première occurence d'un caractère passé en paramètre dans une chaine de caractères elle aussi passée en paramètre. Si le caractère n'est pas présent dans chaîne, la fonction ne fait rien. Par exemple la chaîne `"bonjour a tous"` deviendra `"bnjour a tous"` si on enlève le caractère `'o'`.

6. Écrire une fonction `void delete_useless_space(char *)` qui supprime les espaces inutiles dans une chaîne de caractères passée en paramètre. Par exemple la chaîne `"bonjour␣␣à␣tous"` sera transformée en `"bonjour␣à␣tous"`.

7. On considère la fonction ci-dessous :

```
void mystere(char *s1, char *s2)
{
  while( *s2++ = *s1++ ) {}
}
```

   (a) Quel traitement effectue la fonction `mystere` ?

   (b) Même question en remplaçant la condition dans le `while` par

      i. `*s2++ = *++s1`

      ii. `*++s2 = *s1++`

Exercice 6. Dans cet exercice on va s'intéresser à l'encodage des caractères en `C`.

1. Qu'affiche le code ci-desssous ?

```
#include <stdio.h>
#include <string.h>

int main()
{
  char *s1 = "e";
  printf("longueur de \"e\" : %lu\n",strlen(s1));

  char *s2 = "é";
  printf("longueur de \"é\" : %lu\n",strlen(s2));

  return 0;
}
```

Le codage `UTF-8` est un codage de caractères informatiques conçu pour coder l'ensemble des caractères tout en restant compatible avec la norme `ASCII`. Il s'agit du codage le plus utilisé dans les systèmes GNU/Linux. Son principe est le suivant

- Les nombres entre 0 et 127 correspondent aux caractères de la table `ASCII`.
- Les nombres supérieurs à 128 sont codés sur plusieurs octets ayant chacun leur bit de poids fort non nul :
  - les bits de poids fort du premier octet de la séquence forment une suite de 1 de longueur égale au nombre total d'octets utilisés pour la séquence entière suivie d'un 0
  - les octets suivants ont leurs deux bits de poids fort positionnés à 10

Le codage `UTF-8` permet d'encoder jusqu'à 21 bits d'information, soit plus de deux millions de symboles, sur au plus 4 octets.

Représentation binaire	Signification
0xxxxxxx	1 octet codant 1 à 7 bits
110xxxxx 10xxxxxx	2 octets codant 8 à 11 bits
1110xxxx 10xxxxxx 10xxxxxx	3 octets codant 12 à 16 bits
11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	4 octets codant 17 à 21 bits

2. Donner le codage UTF-8 des caractères suivants :
   (a) `A` (valeur décimale 65)
   (b) `é` (valeur décimale 233)
   (c) `€` (valeur décimale 8364)
   (d) clé de sol (valeur décimale 119070)
3. Écrire un programme qui affiche une clé de sol dans votre terminal.

Pour aller plus loin : écrire une fonction `void utf8_code(char t[4], int v)` qui écrit dans le tableau `t`, le code `UTF-8` correspondant à l'entier `v`. Les éventuels octets non utilisés seront mis à 0.

Exercice 7. On souhaite représenter des dates à l'aide d'une structure.

```
struct date
{
    unsigned int day;
    unsigned int month;
    unsigned int year;
};
```

1. Écrire une fonction `void date_display(struct date)` qui affiche une date sous la forme `25/12/2026`.
2. Écrire une fonction `int is_leap_year(unsigned int)` qui renvoie `1` si l'année passée en paramètre est bissextile et `0` sinon.
3. Écrire une fonction `unsigned int days_in_month(unsigned int, unsigned int)` qui renvoie le nombre de jours d'un mois donné pour une année donnée.
4. Écrire une fonction `int date_compare(struct date, struct date)` qui compare deux dates. La fonction renverra :
   - `0` si les deux dates sont identiques ;
   - `-1` si la première date est antérieure à la seconde ;
   - `1` si la première date est postérieure à la seconde.
5. Écrire une fonction `void date_next_day(struct date *)` qui modifie une date passée par adresse afin de lui ajouter un jour.
6. Écrire une fonction `void date_previous_day(struct date *)` qui réalise l'opération inverse.
7. Écrire une fonction `unsigned int date_difference(struct date, struct date)` qui renvoie le nombre de jours séparant deux dates.

Exercice 8. On souhaite représenter une variable pouvant contenir soit un entier, soit un nombre réel On utilise pour cela une énumération permettant de mémoriser le type de la valeur et une union permettant de mémoriser la valeur elle-même.

```
enum type {INTEGER, REAL};

union value
{
  int integer;
  double real;
};

struct number,
{
  enum type t;
  union value v;
};
```

1. On considère le code suivant :

```
1  struct number n;
2
3  n.t = INTEGER;
4  n.v.integer = 42;
5
6  n.t = REAL;
7  n.v.real = 3.14;
```

   (a) Que contient la variable `v` après l'exécution de ces instructions ?

   (b) Où la valeur `42` est elle située en mémoire à la ligne 4 ? À la ligne 7 ?

   (c) Pourquoi utilise-t-on ici une `union` plutôt qu'une `struct` pour représenter un `number` ?

2. Écrire une fonction `void variable_display(struct number)` qui affiche la valeur contenue dans un paramètre de type `struct number`.

3. Écrire une fonction `struct number make_integer(int)` qui crée et renvoie une variable contenant un entier. Écrire une fonction analogue permettant de créer une variable contenant un nombre réel.

4. Écrire une fonction `void variable_set_integer(struct variable *, int)` qui modifie une variable existante afin qu'elle contienne un entier donné. Écrire une fonction analogue permettant d'affecter un nombre réel.

5. On souhaite maintenant manipuler un tableau de `struct number`. Écrire une fonction `count_type` qui renvoie le nombre de variables d'un type donné dans le tableau de `struct number` passé en paramètre.